

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN - ĐHQG TP HCM
KHOA TOÁN - TIN HỌC
BỘ MÔN ỨNG DỤNG TIN HỌC

THUẬT TOÁN TÌM KIẾM NHỊ PHÂN

THỰC HÀNH CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT
TUẦN 02 - THÁNG 03 NĂM 2026



THÔNG TIN LIÊN HỆ

SAKAI: mcslearn.hcmus.edu.vn

GMAIL: ta.tintth.hcmus@gmail.com

GHI CHÚ:

Tiêu đề mail (bắt buộc):

[CTDL25] [Tiêu đề thư]

VD: [CTDL25] HỎI BÀI

Gửi email kèm giới thiệu họ tên, MSSV
và tên ca học.



NỘI DUNG

01

Mảng thứ tự

1. Insert
2. Search

03

Bài tập tại lớp

1. Ordered Array
 - a. Insert
 - b. Search
2. Binary search
3. ~~Counting Steps~~

02

Thuật toán Tìm kiếm nhị phân

1. Ví dụ cụ thể
2. Pseudocode
3. So sánh với Linear search

04

Bài tập về nhà

1. Ordered Array
2. Binary Search





01

Mảng thứ tự

Ordered Array

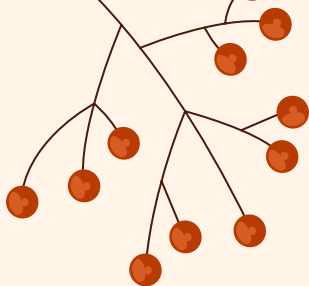




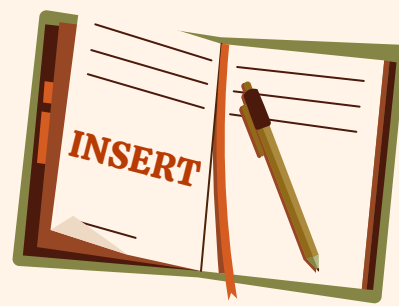
0	1	2	3	4	5
-1	3	12	27	36	100

Mảng thứ tự

Một cấu trúc dữ liệu dạng **mảng thứ tự** là một mảng mà các phần tử của nó được sắp xếp theo một thứ tự nhất định sao cho với bất kỳ chỉ số i ($0 < i < n$) nào thì mệnh đề **$\text{array}[i-1] \leq \text{array}[i] \leq \text{array}[i+1]$** luôn đúng.



Chèn trong Mảng có thứ tự

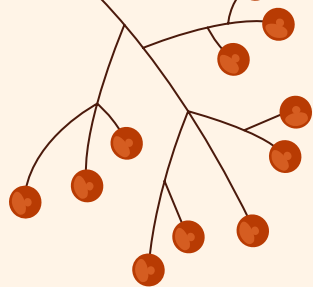


0	1	2	3	4	5
-1	3	12	27	36	100

Thuật toán **INSERT** một phần tử vào mảng có thứ tự:

- **Bước 1:** Xét từ $i = 0$ đến $n - 1$ nếu $\text{array}[i] < \text{value}$ thì $i++$
- **Bước 2:**
 - Nếu $\text{array}[i] \geq \text{value}$ thì **insert** value tại vị trí i
 - Nếu mệnh đề ở bước 1 thỏa khi $i = n - 1$ thì **insert** value ở cuối mảng.





Tìm kiếm trong Mảng thứ tự

0	1	2	3	4	5
-1	3	12	27	36	100

Thuật toán **Linear search** cải tiến:

- **Bước 1:** Xét từ $i = 0$, nếu $\text{array}[i] < \text{value}$ thì $i++$
- **Bước 2:**
 - Nếu $\text{array}[i] == \text{value}$ thì trả i về hàm.
 - Nếu $\text{array}[i] > \text{value}$ thì trả **-1** về hàm.



02

Thuật toán tìm kiếm nhị phân

Binary Search





Ý tưởng:

Cho một **mảng thứ tự** array có kích thước **n**, để tìm kiếm phần tử **value**, ta sẽ thực hiện các bước như sau:

Hàm `binarySearch(array; n; value)`

- **Bước 1:** Gán chỉ số trung vị của mảng cho biến `midPoint`.
- **Bước 2:** Nếu `array[midPoint] == value` thì **return midPoint**.
- **Bước 3:**
 - Nếu `array[midPoint] < value` thì ta không xét tất cả phần tử nằm bên phải mảng. Lúc này nếu mảng bên trái có đúng 1 phần tử thì tức value không xuất hiện trong mảng, ta **return -1**. Ngược lại thì **tái tạo midPoint** là trung vị của mảng bên trái rồi quay về **Bước 2**.
 - Nếu `array[midPoint] > value` thì ta không xét tất cả phần tử nằm bên trái mảng. Lúc này nếu mảng bên phải có đúng 1 phần tử thì tức value không xuất hiện trong mảng, ta **return -1**. Ngược lại thì **tái tạo midPoint** là trung vị của mảng bên phải rồi quay về **Bước 2**.





Ví dụ: Cho biết chỉ số của số 3 trong mảng sau:

1 2 3 4 5 6 7 8 9

Ví dụ: Cho biết chỉ số của số 3 trong mảng sau:

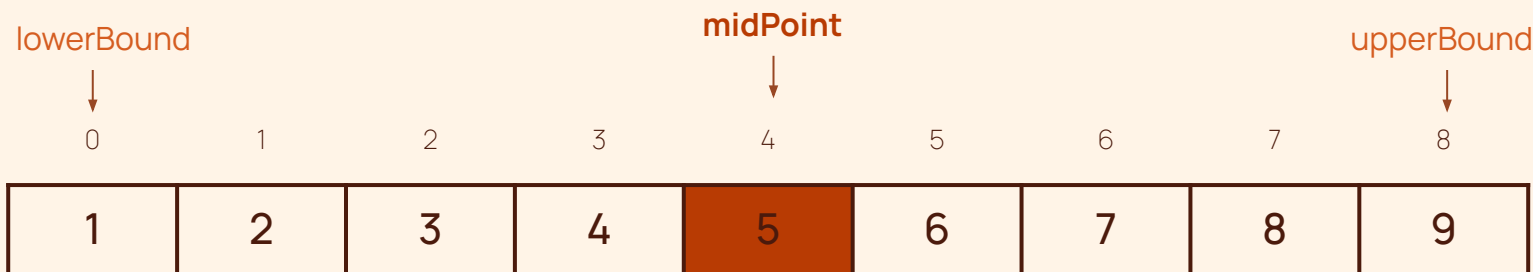
n = 9

value = 3

upperBound = 8

lowerBound = 0

midPoint = 4



Bước 1: Gán chỉ số trung vị của mảng cho biến midPoint.

Bước 2: Nếu `array[midPoint] == value` thì **return** midPoint.

Bước 3:

- Nếu `array[midPoint] < value` thì ta không xét tất cả phần tử nằm bên phải mảng. Lúc này nếu mảng bên trái có đúng 1 phần tử thì tức value không xuất hiện trong mảng, ta **return -1**. Ngược lại thì **tái tạo midPoint** là trung vị của mảng bên trái rồi quay về **Bước 2**.
- Nếu `array[midPoint] > value` thì ta không xét tất cả phần tử nằm bên trái mảng. Lúc này nếu mảng bên phải có đúng 1 phần tử thì tức value không xuất hiện trong mảng, ta **return -1**. Ngược lại thì **tái tạo midPoint** là trung vị của mảng bên phải rồi quay về **Bước 2**.

Ví dụ: Cho biết chỉ số của số 3 trong mảng sau:

n = 9

value = 3

upperBound = 3

lowerBound = 0

midPoint = 1

- **Bước 1:** Gán chỉ số trung vị của mảng cho biến midPoint.
- **Bước 2:** Nếu `array[midPoint] == value` thì `return midPoint`.
- **Bước 3:**
 - Nếu `array[midPoint] < value` thì ta không xét tất cả phần tử nằm bên phải mảng. Lúc này nếu mảng bên trái có đúng 1 phần tử thì tức value không xuất hiện trong mảng, ta `return -1`. Ngược lại thì **tái tạo midPoint** là trung vị của mảng bên trái rồi quay về **Bước 2**.
 - Nếu `array[midPoint] > value` thì ta không xét tất cả phần tử nằm bên trái mảng. Lúc này nếu mảng bên phải có đúng 1 phần tử thì tức value không xuất hiện trong mảng, ta `return -1`. Ngược lại thì **tái tạo midPoint** là trung vị của mảng bên phải rồi quay về **Bước 2**.

$$\text{midPoint} = (\text{upperBound} + \text{lowerBound}) / 2$$

lowerBound	midPoint			upperBound					
0	1	2	3	4	5	6	7	8	12
1	2	3	4	5	6	7	8	9	

Ví dụ: Cho biết chỉ số của số 2 trong mảng sau:

n =

9

value =

3

upperBound =

8

lowerBound =

0

midPoint =

4



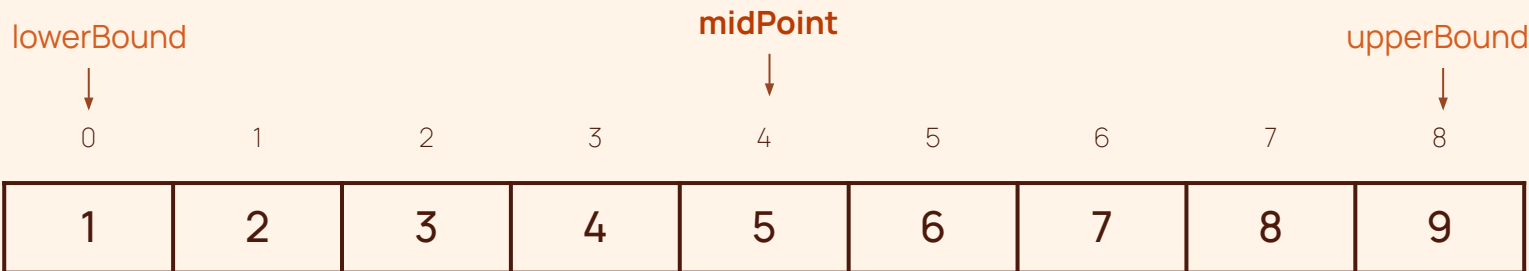
Bước 1: Gán chỉ số trung vị của mảng cho biến **midPoint**.

- **Bước 2:** Nếu **array[midPoint] == value** thì **return midPoint**.

- **Bước 3:**

- Nếu **array[midPoint] < value** thì ta không xét tất cả phần tử nằm bên phải mảng. Lúc này nếu mảng bên trái có đúng 1 phần tử thì tức value không xuất hiện trong mảng, ta **return -1**. Ngược lại thì **tái tạo midPoint** là trung vị của mảng bên trái rồi quay về **Bước 2**.
- Nếu **array[midPoint] > value** thì ta không xét tất cả phần tử nằm bên trái mảng. Lúc này nếu mảng bên phải có đúng 1 phần tử thì tức value không xuất hiện trong mảng, ta **return -1**. Ngược lại thì **tái tạo midPoint** là trung vị của mảng bên phải rồi quay về **Bước 2**.

$$\text{midPoint} = (\text{upperBound} + \text{lowerBound}) / 2$$



Ví dụ: Cho biết chỉ số của số 3 trong mảng sau:

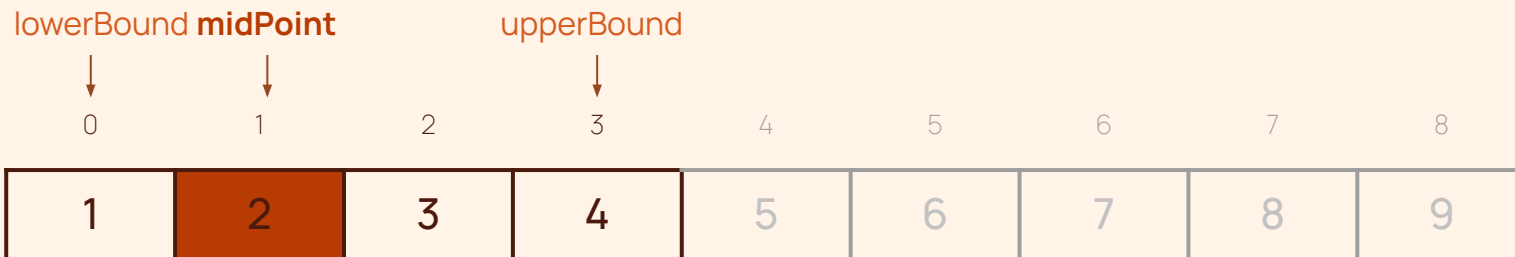
n = 9

value = 3

upperBound = 3

lowerBound = 0

midPoint = 1



Bước 1: Gán chỉ số trung vị của mảng cho biến midPoint.

Bước 2: Nếu `array[midPoint] == value` thì `return midPoint`.

Bước 3:

- Nếu `array[midPoint] < value` thì ta không xét tất cả phần tử nằm bên phải mảng. Lúc này nếu mảng bên trái có đúng 1 phần tử thì tức value không xuất hiện trong mảng, ta `return -1`. Ngược lại thì **tái tạo midPoint** là trung vị của mảng bên trái rồi quay về **Bước 2**.
- Nếu `array[midPoint] > value` thì ta không xét tất cả phần tử nằm bên trái mảng. Lúc này nếu mảng bên phải có đúng 1 phần tử thì tức value không xuất hiện trong mảng, ta `return -1`. Ngược lại thì **tái tạo midPoint** là trung vị của mảng bên phải rồi quay về **Bước 2**.

Ví dụ: Cho biết chỉ số của số 3 trong mảng sau:

n = 9

value = 3

upperBound = 3

lowerBound = 2

midPoint = 2



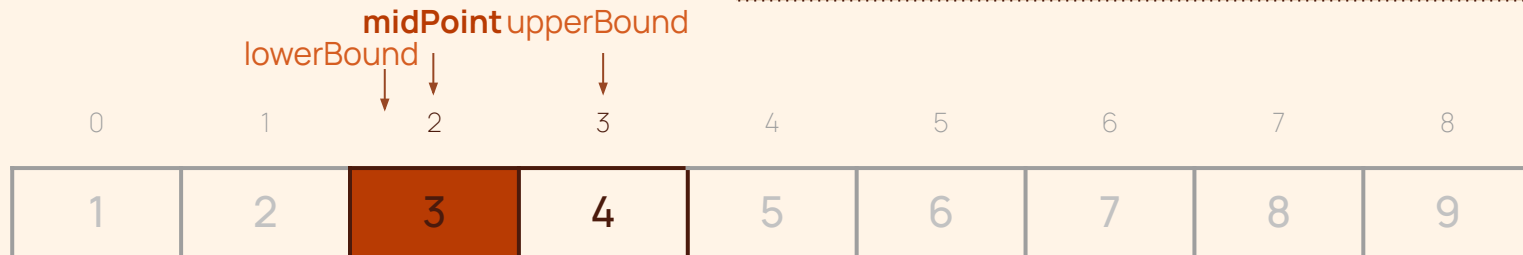
Bước 1: Gán chỉ số trung vị của mảng cho biến midPoint.

Bước 2: Nếu `array[midPoint] == value` thì `return midPoint`.

Bước 3:

- Nếu `array[midPoint] < value` thì ta không xét tất cả phần tử nằm bên phải mảng. Lúc này nếu mảng bên trái có đúng 1 phần tử thì tức value không xuất hiện trong mảng, ta `return -1`. Ngược lại thì **tái tạo midPoint** là trung vị của mảng bên trái rồi quay về **Bước 2**.
- Nếu `array[midPoint] > value` thì ta không xét tất cả phần tử nằm bên trái mảng. Lúc này nếu mảng bên phải có đúng 1 phần tử thì tức value không xuất hiện trong mảng, ta `return -1`. Ngược lại thì **tái tạo midPoint** là trung vị của mảng bên phải rồi quay về **Bước 2**.

Return midPoint;
Số 3 nằm ở chỉ số 2 trong mảng!



Ví dụ: Cho biết chỉ số của số 3 trong mảng sau:

n = 9

value = 3

upperBound = 3

lowerBound = 2

midPoint = 2

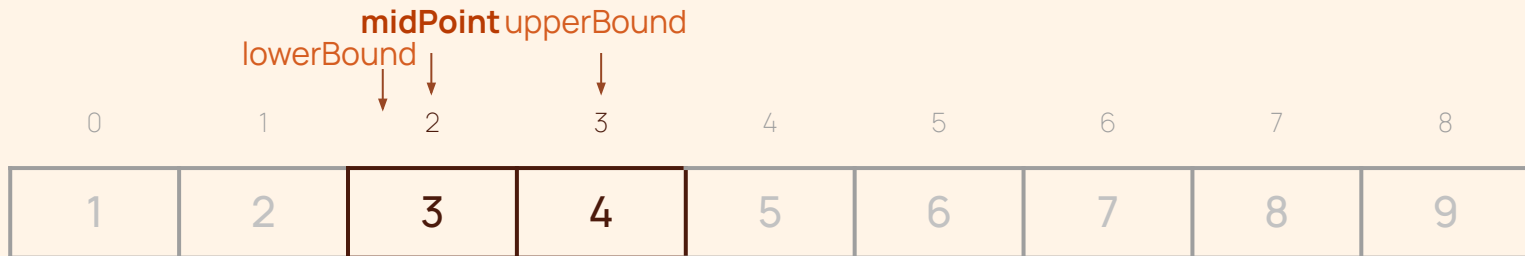
- **Bước 1:** Gán chỉ số trung vị của mảng cho biến midPoint.
- **Bước 2:** Nếu `array[midPoint] == value` thì `return midPoint`.
- **Bước 3:**



Nếu `array[midPoint] < value` thì ta không xét tất cả phần tử nằm bên phải mảng. Lúc này nếu mảng bên trái có đúng 1 phần tử thì tức value không xuất hiện trong mảng, ta `return -1`. Ngược lại thì **tái tạo midPoint** là trung vị của mảng bên trái rồi quay về **Bước 2**.

- Nếu `array[midPoint] > value` thì ta không xét tất cả phần tử nằm bên trái mảng. Lúc này nếu mảng bên phải có đúng 1 phần tử thì tức value không xuất hiện trong mảng, ta `return -1`. Ngược lại thì **tái tạo midPoint** là trung vị của mảng bên phải rồi quay về **Bước 2**.

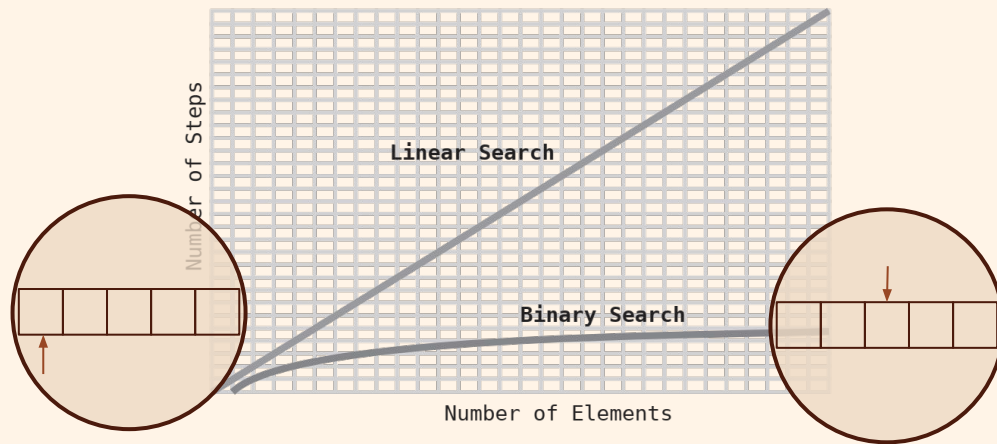
$$\text{midPoint} = (\text{upperBound} + \text{lowerBound}) / 2$$



Pseudocode cho thuật toán tìm kiếm nhị phân

```
def binary_search(array, value)
  lower_bound = 0
  upper_bound = array.length - 1
  while lower_bound <= upper_bound do
    midpoint = (upper_bound + lower_bound) / 2
    value_at_midpoint = array[midpoint]
    if value < value_at_midpoint
      upper_bound = midpoint - 1
    elsif value > value_at_midpoint
      lower_bound = midpoint + 1
    elsif value == value_at_midpoint
      return midpoint
    end
  end
end
return nil
end
```

So sánh



Linear search

- Kiểu dữ liệu phù hợp: mảng bất kỳ.
- Số bước tìm kiếm giá trị có trong mảng: nhiều
- Số bước tìm kiếm giá trị không có trong mảng: rất nhiều.

Binary search

- Kiểu dữ liệu phù hợp: mảng thứ tự.
- Số bước tìm kiếm giá trị có trong mảng: rất ít
- Số bước tìm kiếm giá trị không có trong mảng: rất ít



03

Bài tập tại lớp

Ordered Array

Binary Search

Bài tập tại lớp 1



Cho một mảng thứ tự kiểu nguyên nhập từ bàn phím và một số nguyên. Hãy xây dựng hàm **insertOrderedArray** với chức năng chèn một số nguyên vào mảng thứ tự đã cho trên. Viết chương trình C để thử nghiệm hàm đó.

INPUT:

- kích thước mảng **n**
- mảng số nguyên **array**
- số nguyên **value** cần chèn

OUTPUT: Mảng **array** sau khi chèn.

TEST CASE BÀI TẬP TẠI LỚP 01

TEST	INPUT	OUTPUT
1	6 -1 3 12 27 36 100 4	-1 3 4 12 27 36 100
2	8 1 2 3 3 4 9 15 22 3	1 2 3 3 3 4 9 15 22
3	9 -17 -16 -10 3 9 10 126 130 290 -100	-100 -17 -16 -10 3 9 10 126 130 290

Bài tập tại lớp 02



Cho một mảng thứ tự kiểu nguyên nhập từ bàn phím và một số nguyên. Hãy xây dựng hàm **orderedLinearSearch** với chức năng tìm số nguyên đó trong mảng thứ tự.

INPUT:

- kích thước mảng **n**
- mảng số nguyên **array**
- số nguyên **value** cần tìm

OUTPUT: Chỉ số đầu tiên tìm thấy trong mảng.

TEST CASE BÀI TẬP TẠI LỚP 02

TEST	INPUT	OUTPUT
1	6 -1 3 12 27 36 100 12	2
2	8 1 2 3 3 4 9 15 22 3	2
3	9 -17 -16 -10 3 9 10 126 130 290 291	Not found!

Bài tập tại lớp 3



Cho một mảng thứ tự kiểu nguyên nhập từ bàn phím và một số nguyên. Hãy xây dựng hàm **binarySearch** với chức năng tìm số nguyên đó trong mảng.

INPUT:

- kích thước mảng **n**
- mảng số nguyên **array**
- số nguyên **value** cần tìm

OUTPUT: Chỉ số tìm thấy trong mảng.

TEST CASE BÀI TẬP TẠI LỚP 03

TEST	INPUT	OUTPUT
1	6 -1 3 12 27 36 100 12	2
2	8 1 2 3 3 4 9 15 22 3	3
3	9 -17 -16 -10 3 9 10 126 130 290 291	Not found!

Bài tập tại lớp 4: COUNTING STEPS



Hãy thêm biến **compsLinear** và **compsBinary** để đếm số lần so sánh giữa `array[i]` và `value` lần lượt trong hai hàm **orderedLinearSearch** và **binarySearch** đã làm ở bài tập 3 và bài tập 4. Có nhận xét gì?

TEST	INPUT	OUTPUT
1	6 -1 3 12 27 36 100 12	stepLinear = 3 stepBinary = 1
2	8 1 2 3 3 4 9 15 22 3	stepLinear = 3 stepBinary = 1
3	9 -17 -16 -10 3 9 10 126 130 290 11	stepLinear = 7 stepBinary = 4

04

Bài tập về nhà

Ordered Array

Binary Search





Bài 1: Hãy làm lại bài tập tại lớp 1,2,3 với kiểu dữ liệu số thực.

Bài 2: Lập trình một chương trình để thực hiện tất cả yêu cầu sau:

Cho một chuỗi ký tự và một ký tự **value**.

- a) Hãy kiểm tra chuỗi ký tự đó đã được sắp xếp theo thứ tự tăng dần chưa? Nếu chưa thì không làm câu **b** và câu **c**.
 - b) Dùng và biến đổi hàm **linearSearch** có được từ **Bài tập tại lớp 2**, hãy tìm chỉ số của **value** trong mảng.
 - c) Dùng và biến đổi hàm **binarySearch** có được từ **Bài tập tại lớp 2**, hãy tìm chỉ số của **value** trong mảng và so sánh kết quả với câu **b**.
- 

Bài 3: Cho một chuỗi ký tự có 25 ký tự, mỗi ký tự là một chữ cái thường và không xuất hiện quá 1 lần trong chuỗi. Hãy phát hiện chữ cái còn thiếu để hoàn thành một chuỗi có 26 ký tự đầy đủ theo thứ tự Alphabet.



TEST CASE BÀI TẬP VỀ NHÀ 3

TEST	INPUT	OUTPUT
1	bcdefghijklmnopqrstuvwxyz	a at 0 is missing! abcdefghijklmnopqrstuvwxyz
2	abcdefghijklmnopqrstuvwxyz	j at 9 is missing! abcdefghijklmnopqrstuvwxyz
3	abcdefghijklmnopqrstuvwxy	z at 25 is missing! abcdefghijklmnopqrstuvwxy z

Câu hỏi phụ: Em đã dùng những thuật toán nào để giải quyết vấn đề này? Hãy sử dụng **binarySearch** để phát hiện chữ cái còn thiếu rồi đánh giá thuật toán **binarySearch** có phù hợp với vấn đề này hay không?

The slide features a light beige background decorated with various autumn-themed illustrations. On the left side, there are several green leaves, a large orange maple leaf, and a branch with small red berries. In the center-left, there is a green and orange striped sock with white floral patterns, and below it, an open notebook with a pen resting on it. To the right of the notebook is another green and orange striped sock with white floral patterns. On the right side, there are more orange and green leaves, and a small branch with red berries. The text "THE END!" is written in a large, bold, orange serif font. Below it, the text "Do you have any questions?" is written in a smaller, bold, orange sans-serif font. Below that, the text "Email us at" is written in a smaller, orange sans-serif font, followed by the email address "tin.tth.hcmus@gmail.com" in a smaller, orange sans-serif font. The number "30" is visible in the bottom right corner, partially obscured by a leaf.

THE END!

Do you have any questions?

Email us at

tin.tth.hcmus@gmail.com